

COMPANION
to

ChatGPT Teaches TikZ

Appendix B: The TikZ Library Guide

Contents

B.1	Core TikZ and TikZ Libraries	4
B.2	How Many TikZ Libraries Are There?	5
B.3	Libraries Used in This Book	6
B.4	Additional Relevant Libraries	12
B.5	Libraries Not Covered in the Main Chapters	13
B.6	Libraries Every Mathematician Should Know	20
B.7	Choosing a Library by Task	22
B.8	TikZ Libraries and External Packages	22
B.9	A Recommended Loading Policy	24
B.10	When a Library Appears Not to Load	25
B.11	Alphabetical Inventory of <code>tikzlibrary</code> Files	26
B.12	Final Advice	27

TikZ is not a single, indivisible collection of drawing commands. It has a small core and a large collection of optional libraries. Each library adds commands, keys, styles, shapes, or specialized drawing methods.

A library is normally loaded in the preamble with a command of the form

Code

```
\usetikzlibrary{positioning}
```

Several libraries can be loaded at once.

Code

```
\usetikzlibrary{
  arrows.meta,
  calc,
  intersections,
  positioning
}
```

The library system is important for three reasons.

First, it keeps the TikZ core manageable. A document that draws only simple lines and circles does not need machinery for electrical circuits, mind maps, Petri nets, or three-dimensional data visualization.

Second, a library tells the reader what kind of work a diagram performs. For example, the command

Code

```
\usetikzlibrary{intersections}
```

immediately signals that named paths and computed intersections may appear later in the document.

Third, library names provide useful search terms. When a reader encounters a problem involving relative placement, coordinate calculations, braces, patterns, or modern arrow tips, the appropriate library name points toward the relevant part of the TikZ manual.

Principle

Load a TikZ library because a diagram needs a feature supplied by that library, not merely because the library might be useful someday.

B.1 Core TikZ and TikZ Libraries

The package

Code

```
\usepackage{tikz}
```

loads TikZ itself. It provides the basic picture environment, coordinates, paths, nodes, drawing operations, fills, colors, transformations, and many common options.

A TikZ library is an optional extension loaded with

Code

```
\usetikzlibrary
```

For example, the core syntax can draw two nodes, but the `positioning` library supplies the especially convenient syntax

Code

```
\node (B) [right=of A] {$B$};
```

Similarly, basic coordinates belong to TikZ, while the `calc` library provides coordinate calculations such as

Code

```
($ (A)!0.5!(B)$)
```

The distinction is therefore not between “TikZ commands” and “non-TikZ commands.” Libraries are part of the TikZ system, but they are loaded only when needed.

B.2 How Many TikZ Libraries Are There?

There is no completely universal answer.

One count may include only the principal libraries named in the manual. Another may count every separately loadable sublibrary, such as `decorations.pathmorphing`, `datavisualization.barcharts`, and `circuits.logic.IEC`. A third may also include compatibility libraries, umbrella libraries, and older libraries retained for existing documents.

In the TeX installation used while preparing this appendix, the TikZ library directory contains **74 separately named library files**. That number should not be treated as permanent. PGF/TikZ develops over time, and another TeX distribution or another release may contain a different total.

External packages should not be included in this count. Packages such as `pgfplots`, `tikz-cd`, `circuitikz`, and `forest` are closely related to TikZ, but they are separate packages rather than TikZ libraries.

Principle

The exact total is less important than knowing which library solves the problem at hand. Most mathematical diagrams require only a small part of the complete TikZ library collection.

B.3 Libraries Used in This Book

The following libraries form the principal working collection used in this book. The chapter note for each library identifies its principal introduction or application in the main text.

`arrows.meta`

Purpose. Provides the modern system of configurable arrow tips.

Typical use.

Code

```
\usetikzlibrary{arrows.meta}

\draw[-{Stealth}] (A)--(B);
```

This library replaces many uses of the older `arrows` library. It supports arrow tips such as `Stealth`, `Latex`, `Triangle`, and `Circle`, together with options controlling their length, width, scale, and appearance.

Status in this book. Used.

Principal chapter. Chapter 10.

backgrounds

Purpose. Provides convenient access to a background layer.

Typical use.

Code

```
\usetikzlibrary{backgrounds}

\begin{scope}[on background layer]
  \fill[gray!15] (-1,-1) rectangle (4,2);
\end{scope}
```

It is useful when a diagram contains highlighting regions, panels, frames, or other material that must appear behind the principal objects.

Status in this book. Used.

Principal chapter. Chapter 24.

calc

Purpose. Performs coordinate calculations.

Typical use.

Code

```
\usetikzlibrary{calc}

\coordinate (M) at ($(A)!0.5!(B)$);
```

The library can form midpoints, weighted points, coordinate sums and differences, projections, and shifted coordinates. It is especially valuable in geometric constructions.

Status in this book. Used.

Principal chapter. Chapter 8.

`decorations.pathmorphing`

Purpose. Changes the visible form of a path.

Typical use.

Code

```
\usetikzlibrary{decorations.pathmorphing}

\draw[
  decorate,
  decoration={coil}
]
(0,0)--(5,0);
```

Among its common decorations are coils, zigzags, waves, and snake-like paths. These are useful for springs, flexible connectors, vibrations, and special boundaries.

Status in this book. Used.

Principal chapter. Chapter 13.

`decorations.pathreplacing`

Purpose. Replaces a path by a related decorative form.

Typical use.

Code

```
\usetikzlibrary{decorations.pathreplacing}

\draw[
```


Code

```
    decorate,  
    decoration={brace,amplitude=5pt}  
  ]  
(0,0)--(4,0);
```

Its best-known application is the brace decoration. It is useful for grouping objects and attaching explanatory labels.

Status in this book. Used.

Principal chapter. Chapter 13.

intersections

Purpose. Computes intersections of named paths.

Typical use.

Code

```
\usetikzlibrary{intersections}  
  
\path[name path=lineA] (0,0)--(4,4);  
\path[name path=lineB] (0,4)--(4,0);  
  
\path[  
  name intersections={  
    of=lineA and lineB,  
    by=0  
  }  
];
```

This library is central to geometric constructions in which points are defined by the intersections of lines, circles, or curves.

Status in this book. Used.

Principal chapter. Chapters 15, 16, and 26.

matrix

Purpose. Creates matrices of nodes.

Typical use.

Code

```
\usetikzlibrary{matrix}

\matrix[
  matrix of nodes,
  row sep=4mm,
  column sep=6mm
]
{
  $a$ & $b$ \\
  $c$ & $d$ \\
};
```

The library is useful not only for mathematical matrices but also for grids, arrays, tables of nodes, adjacency displays, and diagram layouts.

Status in this book. Used.

Principal chapter. Chapter 12.

patterns

Purpose. Fills regions with repeating graphical patterns.

Typical use.

Code

```
\usetikzlibrary{patterns}

\fill[
```

Code

```
    pattern=north east lines  
]  
(0,0) rectangle (3,2);
```

Patterns allow regions to remain distinguishable in grayscale printing and can communicate different mathematical classes without relying on color alone.

Status in this book. Available through the shared style, but not developed in a main-text example.

positioning

Purpose. Places nodes relative to other nodes.

Typical use.

Code

```
\usetikzlibrary{positioning}  
  
\node (A) {$A$};  
\node (B) [right=of A] {$B$};  
\node (C) [below=of A] {$C$};
```

This is one of the most useful TikZ libraries. It replaces fragile manual coordinate calculations with relationships such as `right=of`, `below=of`, and `above left=of`.

Status in this book. Used.

Principal chapter. Chapter 6.

B.4 Additional Relevant Libraries

The following libraries fit the aims of the book. Some are used in the final manuscript; the remaining entries are useful candidates for a future expanded treatment.

`chains`

The `chains` library places a sequence of nodes along a chain. It can simplify flowcharts, timelines, and repeated layouts. Its syntax is powerful, but it should be introduced only if the manuscript contains a substantial example that benefits from it.

Status in this book. Candidate.

`fit`

The `fit` library creates a node that encloses a collection of existing nodes or coordinates. It is useful for grouping related objects, drawing panels, and marking subsystems.

Status in this book. Used in Chapters 24 and 28.

`patterns.meta`

The `patterns.meta` library provides a newer and more flexible pattern mechanism than the traditional `patterns` library. It is appropriate when patterns must be customized.

Status in this book. Candidate.

quotes

The `quotes` library supplies a compact syntax for labels on edges and paths. It can make some diagrams shorter, but the abbreviated syntax should be introduced only after readers understand ordinary nodes and edge labels.

Status in this book. Candidate.

shapes.geometric

The `shapes.geometric` library adds diamonds, regular polygons, trapezia, stars, and other geometric node shapes. It is particularly useful in flowcharts and structured diagrams.

Status in this book. Loaded by the main document; not treated as a separate chapter topic.

B.5 Libraries Not Covered in the Main Chapters

A library may be omitted because it is highly specialized, because it requires a separate treatment, because a newer library is preferred, or because it lies outside the mathematical focus of this book. Omission does not imply that a library is unimportant.

Three-Dimensional Drawing and Views

3d Provides coordinate systems and tools for elementary three-dimensional drawing.

perspective Supports perspective transformations and vanishing-point constructions.

views Supports reusable views and transformations of parts of a picture.

These libraries are useful, but convincing three-dimensional illustration requires a chapter of its own.

Angles and Geometric Annotations

angles Draws and labels angles formed by lines or coordinates.

quotes Provides concise labeling syntax and works especially well with **angles**.

through Constructs nodes whose shapes pass through specified points.

The **angles** library would be a natural addition to an expanded chapter on Euclidean constructions.

Animation

animations Provides an animation system for changing attributes over time.

Animation depends strongly on output format and viewer support. It is therefore outside the scope of a book centered on static mathematical diagrams.

Automata, Petri Nets, and Mind Maps

automata Supplies styles and conventions for finite-state automata.

petri Supplies styles for Petri nets.

mindmap Supports radial mind maps and concept maps.

These libraries are important in their own subject areas but are too specialized for the main sequence of this book.

Arrow Systems and Path Bending

arrows Provides the older TikZ arrow-tip system. New documents should normally use **arrows.meta**.

bending Improves the way arrow tips follow curved paths.

topaths Provides specialized constructions for the **to** path operation.

The **bending** library becomes important when arrow tips must follow sharply curved paths accurately.

Babel Compatibility

babel Helps TikZ cooperate with active punctuation introduced by language packages.

This is primarily a compatibility library. It may be essential in multilingual documents even though it does not create a new kind of diagram.

Calendars

calendar Creates calendar layouts and date-based diagrams.

Calendar construction is useful, but it does not belong to the central mathematical drawing progression of this book.

Circuits

circuits Provides the general TikZ circuit framework.

circuits.ee Provides electrical-engineering circuit symbols.

`circuits.ee.IEC` Provides electrical symbols following IEC conventions.

`circuits.logic` Provides the general logic-circuit framework.

`circuits.logic.CDH` Provides logic gates in the CDH style.

`circuits.logic.IEC` Provides logic gates following IEC conventions.

`circuits.logic.US` Provides logic gates in the traditional United States style.

Readers who need extensive circuit diagrams may prefer the separate `circuitikz` package.

Data Visualization

`datavisualization` Provides a general data-visualization framework.

`datavisualization.3d` Extends data visualization to three dimensions.

`datavisualization.barcharts` Provides bar-chart tools.

`datavisualization.formats.functions` Accepts mathematical functions as data sources.

`datavisualization.polar` Provides polar data visualizations.

`datavisualization.sparklines` Creates small inline data graphics.

This family is substantial enough to require a separate treatment. For conventional mathematical plotting, many readers will instead use the separate `pgfplots` package.

Decoration Libraries

`decorations` Acts as an umbrella for the decoration system.

`decorations.footprints` Places footprint-like marks along a path.

`decorations.fractals` Creates certain fractal path decorations.

decorations.markings Places marks, nodes, or arrows at specified positions along a path.

decorations.shapes Places repeated shapes along a path.

decorations.text Places text along a path.

The main chapters use only the decoration libraries needed for coils, zigzags, waves, and braces. The remaining libraries are valuable extensions.

Entity–Relationship Diagrams

er Provides styles for entity–relationship diagrams.

This library is specialized for database modeling.

Fadings, Shadings, and Shadows

fadings Controls transparency through fading masks.

shadings Provides smooth color transitions.

shadows Adds shadows to nodes and paths.

These libraries can create attractive effects, but decorative effects should be used cautiously in mathematical exposition.

Folding

folding Constructs foldable shapes and related diagrams.

This is a specialized library and is not needed in the main progression.

Mathematical Computation

fixedpointarithmetic Provides fixed-point arithmetic tools.

fpu Provides floating-point calculations.

math Provides mathematical functions and parsing tools.

These libraries support calculations behind diagrams. They are generally encountered when programming more advanced TikZ constructions.

Graphs

graphs Provides a compact syntax for specifying graphs.

graphs.standard Provides standard graph constructions and patterns.

Automatic graph drawing is related but is supported by the PGF graph drawing system and often requires LuaLaTeX. A full account belongs in an advanced volume or a dedicated chapter.

L-Systems, Turtles, and Fractals

lindenmayersystems Creates figures from Lindenmayer systems.

turtle Provides turtle-graphics commands.

These libraries are useful for recursive figures, plants, fractals, and educational graphics.

Plotting Support

plothandlers Provides handlers that control how plot coordinates are connected.

plotmarks Provides marks used at data points.

These libraries support TikZ plotting. For extensive plots and axes, **pgfplots** is generally the more appropriate tool.

Scopes and Reusable Structure

scopes Provides compact scope-related syntax.

rdf Supports semantic information associated with graphics.

These are advanced structural tools rather than basic drawing libraries.

Shape Libraries

shapes Acts as an umbrella for several shape collections.

shapes.arrows Provides arrow-shaped nodes.

shapes.callouts Provides callout and annotation shapes.

shapes.gates.logic.IEC Provides IEC logic-gate node shapes.

shapes.gates.logic.US Provides traditional United States logic-gate node shapes.

shapes.misc Provides assorted additional shapes.

shapes.multipart Provides nodes divided into several separately addressable parts.

shapes.symbols Provides symbolic node shapes.

The separate **shapes.geometric** library is a strong candidate for inclusion because geometric shapes occur frequently in flowcharts and mathematical diagrams.

Older and Specialized Path Libraries

snakes Provides an older path-decoration mechanism. Modern documents should normally use the appropriate **decorations.*** library.

svg.path Allows paths written in SVG path syntax.

The **snakes** library is retained mainly for compatibility with older documents.

Spy Graphics

spy Creates magnified views of selected parts of a picture.

This is useful when a detailed portion of a large diagram must be enlarged without redrawing it separately.

B.6 Libraries Every Mathematician Should Know

A mathematician does not need to memorize the complete catalog. The following compact collection covers a very large proportion of ordinary mathematical diagrams.

positioning For placing nodes relative to one another.

calc For midpoints, weighted points, shifts, and coordinate arithmetic.

arrows.meta For modern and configurable arrow tips.

matrix For matrices, grids, arrays, and structured node layouts.

intersections For constructing points from intersecting paths.

decorations.pathmorphing For coils, zigzags, waves, and related path effects.

decorations.pathreplacing For braces and replacement decorations.

patterns For patterned regions, especially in grayscale documents.

backgrounds For material that must appear behind the main diagram.

fit For enclosing and grouping existing nodes.

shapes.geometric For diamonds, polygons, stars, and related node shapes.

Principle

Learn libraries by using them. A short list of familiar libraries is more valuable than a long list of names that have never appeared in a working diagram.

B.7 Choosing a Library by Task

Task	Library
Place one node relative to another	<code>positioning</code>
Compute a midpoint or shifted coordinate	<code>calc</code>
Use modern arrow tips	<code>arrows.meta</code>
Build a matrix or grid of nodes	<code>matrix</code>
Find where two paths meet	<code>intersections</code>
Draw a coil, wave, snake, or zigzag	<code>decorations.</code> <code>pathmorphing</code>
Draw a brace	<code>decorations.</code> <code>pathreplacing</code>
Fill a region with a pattern	<code>patterns</code>
Draw behind the main picture	<code>backgrounds</code>
Enclose several nodes in one region	<code>fit</code>
Use diamonds, polygons, or stars	<code>shapes.geometric</code>
Place text along a path	<code>decorations.text</code>
Place marks or arrows along a path	<code>decorations.</code> <code>markings</code>
Magnify part of a diagram	<code>spy</code>
Draw an automaton	<code>automata</code>
Draw a Petri net	<code>petri</code>
Draw a mind map	<code>mindmap</code>
Draw a calendar	<code>calendar</code>
Create a logic circuit	<code>circuits.logic.*</code>
Create an electrical circuit	<code>circuits.ee.*</code>

B.8 TikZ Libraries and External Packages

TikZ libraries extend TikZ from within the PGF/TikZ distribution. External packages build larger systems on top of TikZ or use TikZ as their graphical foundation.

The distinction matters because the loading commands are different.

A TikZ library is loaded with

Code

```
\usetikzlibrary{matrix}
```

An external package is loaded with

Code

```
\usepackage{tikz-cd}
```

The following external packages are especially relevant to mathematicians.

pgfplots

Creates publication-quality function plots, data plots, axes, legends, and three-dimensional plots. It should normally be preferred when the main purpose of a picture is plotting numerical or functional data.

tikz-cd

Creates commutative diagrams using a matrix-like syntax. It is often more convenient than building a large commutative diagram directly with ordinary TikZ nodes and arrows.

forest

Creates trees using a concise hierarchical syntax. It is particularly effective for large rooted trees, syntax trees, and proof trees.

circuitikz

Creates electrical and electronic circuit diagrams. It provides a much larger specialized vocabulary than the basic TikZ circuit libraries.

Warning

Do not load `pgfplots` or `tikz-cd` with `\usetikzlibrary`. They are packages, not TikZ libraries.

B.9 A Recommended Loading Policy

There are two reasonable ways to load libraries.

The first is to load only the libraries required by the current document.

Code

```
\usetikzlibrary{
  calc,
  intersections,
  positioning
}
```

This makes dependencies visible and reduces the chance that an example works only because an unrelated file loaded a library indirectly.

The second is to maintain a common style file for a large book and load the libraries used throughout the manuscript there.

Code

```
\usetikzlibrary{
  arrows.meta,
  backgrounds,
  calc,
  decorations.pathmorphing,
  decorations.pathreplacing,
  intersections,
  matrix,
  patterns,
  positioning
}
```

For a book, the second policy is convenient, but every chapter should still tell the reader which library supplies a newly introduced feature. Otherwise a copied example may fail when placed in a smaller document.

Warning

A library loaded in the book's style file may be invisible to the reader. When an example depends on a library, identify that library in the surrounding discussion even if the manuscript already loads it globally.

B.10 When a Library Appears Not to Load

A command may fail even when the library name appears in a style file. Several different problems can produce this impression.

1. The style file may not actually be loaded by the document.

2. The library-loading command may occur inside a conditional branch that is not executed.
3. An earlier error may stop TeX before the library-loading command is read.
4. The library name may be misspelled.
5. The feature may belong to a different library than expected.
6. The document may be using another copy of the style file found earlier in TeX's search path.
7. Auxiliary files or an editor's build configuration may cause the wrong main file to be compiled.

A useful diagnostic test is to place the required command temporarily in the main source file.

Code

```
\usetikzlibrary{decorations.pathmorphing}
```

If the example then works, the TikZ library itself is available. The remaining problem is the document's loading structure, file search path, or compilation setup.

Principle

When diagnosing a missing-library error, first prove that the smallest possible document can load the library successfully. Then investigate the book's class file, style file, and main source file.

B.11 Alphabetical Inventory of tikzlibrary Files

The following inventory records the 74 files whose names have the form `tikzlibrary...code.tex` in the TikZ library tree of the TeX installation used to prepare this companion, including its subdirectories. It is a map of that file tree, not a list that readers are

expected to memorize. Some names accepted by `\usetikzlibrary`, including `arrows.meta` and `external`, are supplied through other PGF library mechanisms or locations and therefore are not part of this particular 74-file inventory.

3d, angles, animations, arrows, automata, babel, backgrounds, bending, calc, calendar, chains, circuits, circuits.ee, circuits.ee.IEC, circuits.logic, circuits.logic.CDH, circuits.logic.IEC, circuits.logic.US, datavisualization, datavisualization.3d, datavisualization.barcharts, datavisualization.formats.functions, datavisualization.polar, datavisualization.sparklines, decorations, decorations.footprints, decorations.fractals, decorations.markings, decorations.pathmorphing, decorations.pathreplacing, decorations.shapes, decorations.text, er, fadings, fit, fixedpointarithmetic, folding, fpu, graphs, graphs.standard, intersections, lindenmayersystems, math, matrix, mindmap, patterns, patterns.meta, perspective, petri, plohandlers, plotmarks, positioning, quotes, rdf, scopes, shadings, shadows, shapes, shapes.arrows, shapes.callouts, shapes.gates.logic.IEC, shapes.gates.logic.US, shapes.geometric, shapes.misc, shapes.multipart, shapes.symbols, snakes, spy, svg.path, through, topaths, trees, turtle, and views.

A future TikZ release may add, rename, reorganize, or retire libraries. The TikZ and PGF manual belonging to the reader's installed TeX distribution remains the definitive reference.

B.12 Final Advice

The library system can look intimidating because the complete list is long. In practice, the reader should proceed one problem at a time.

When a diagram needs relative placement, learn **positioning**. When it needs a midpoint, learn **calc**. When it needs an intersection, learn **intersections**. When it needs a brace, learn **decorations.pathreplacing**.

After several chapters, the same small group of libraries will recur. At that point, loading them will no longer feel like an additional technical step. They will simply be part of the reader's TikZ vocabulary.

Principle

TikZ libraries are not obstacles placed between the reader and the diagram. They are organized collections of solutions.